

UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Identification (Entity Authentication)

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Identification

- Also referred to as Entity Authentication
- Techniques designed to allow one party (the verifier or authenticator) to gain assurances that the identity of another (the prover, claimant, or supplicant) is as declared
  - **preventing impersonation**
- A major difference between entity authentication and message authentication is that
  - **message authentication itself provides no timeliness guarantees with respect to when a message was created**
  - **whereas entity authentication involves proof of a claimant's identity through actual communications with an associated verifier during execution of the protocol itself**
    - provide assurances only at the particular instant in time of successful protocol completion
      - If ongoing assurances are required, additional measures may be necessary

Basis of identification

- Entity authentication techniques may be divided into three main categories, depending on which of the following the security is based:
  - **1. something known**
    - e.g. standard passwords (sometimes used to derive a symmetric key), Personal Identification Numbers (PINs), secret or private keys
  - **2. something possessed**
    - this is typically a physical accessory, as a “passport”
    - e.g. cards like magnetic-striped cards, chip cards (also called smart cards or IC cards), and hand-held customized calculators (passwd generators) which provide time-variant passwords
  - **3. something inherent to a human individual**
    - this category includes methods which make use of human physical characteristics and involuntary actions (biometrics), such as handwritten signatures, fingerprints, voice, retinal patterns, and dynamic keyboarding characteristics
    - these techniques are not further discussed

Characteristics of identification protocols

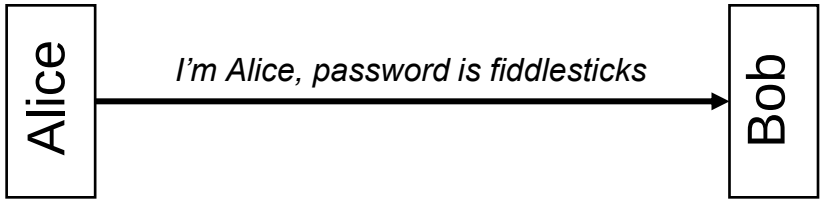
- Direction of the identification (reciprocity):
  - **unilateral identification**
    - only one party proves its identity to the other party
  - **mutual identification**
    - both parties may corroborate their identities to the other
- Computational and/or communication efficiency:
  - **computational efficiency**
    - the number of operations required to execute a protocol
  - **communication efficiency**
    - this includes the number of passes (message exchanges) and the bandwidth required (total number of bits transmitted)

Characteristics of identification protocols (cont.)

- Real-time involvement of a third party (if any):
  - an on-line trusted third party to distribute symmetric keys to communicating entities for authentication purposes; or
  - an on-line trusted third party to verify the authentication information sent by the supplicant, or
  - an on-line (untrusted) directory service for distributing public-key certificates
- Storage of secrets
  - **which information must be stored to perform verification**
    - in case of symmetric secrets
      - in clear or encrypted/hashed value
    - in case of public keys, they are not secret, however:
      - how obtaining public key of the peer-entity
      - how storing public key of the peer-entity
  - **where secrets are maintained/stored**
    - RAM
    - local disks
    - hardware tokens

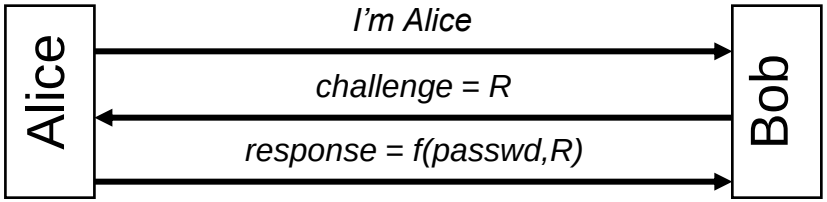
Basic authentication scheme

- Conventional (basic) authentication schemes consist in sending a fixed (time-invariant) symmetric secret
  - **shared secret between the user and system**
    - like a key or a password
  - **thus fall under the category of symmetric-key techniques**
- For example, to gain access to a system resource the user enters a *{user id, password}* pair
  - **weak authentication if used through an insecure channel**
  - **it is not associated to a given time (time-invariant)**
  - **the verifier can store the hash of the secret**

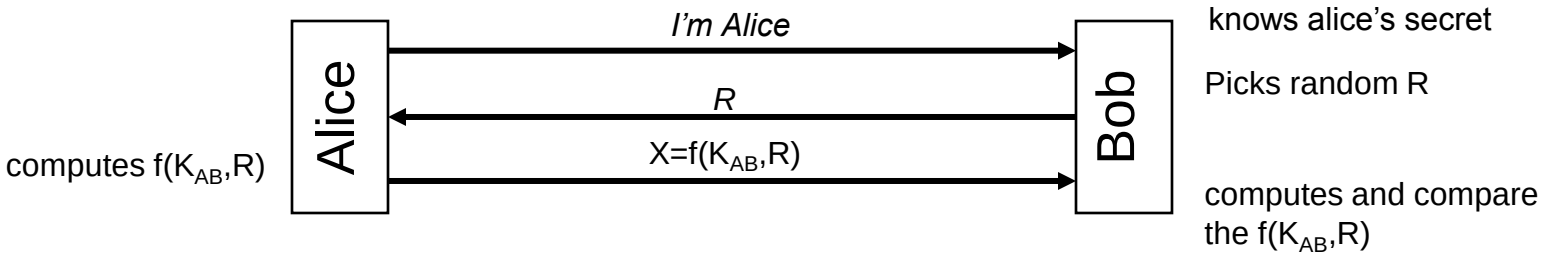


Challenge-response authentication

- The idea is that one entity (the claimant) “proves” its identity to another entity (the verifier) by demonstrating knowledge of a secret, without revealing the secret itself to the verifier
- This is done by providing a response to a *time-variant* challenge, where the response depends on both the challenge and the entity’s secret
  - ***time-variant* challenge is used to counteract replay and interleaving attacks**
    - a random number, text string, or bit string chosen by one entity
    - a sequence number, incremented sequentially, or a timestamp referring to a given time interval, optionally sent during the exchange
      - a validity interval of values may be considered
  - **the response is verified by using the same entity’s secret OR other information associated to the secret**
    - e.g. hash value, public key, etc.

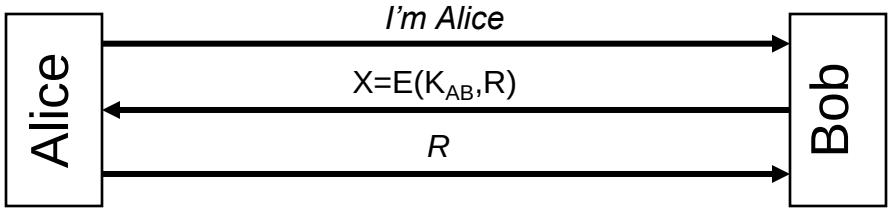


Authentication with symmetric key



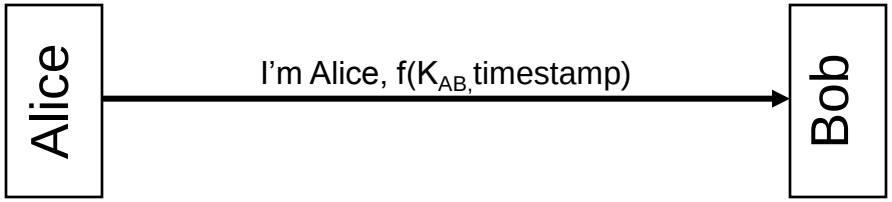
- X= f(K_{AB},R) cryptographic function
  - **not require reversible cryptography**
    - can be a one-way
  - **examples**
    - X= MAC(K_{AB},R), X= H($secret_{AB}$ || R), X= E(K_{AB},R)
- drawbacks:
  - **an eavesdropper could mount an off-line password guessing attack**
  - **It requires that the authenticator maintain a copy of the password/key**
    - some who read the Bob's passwd-database (the authenticator) can later impersonate Alice (the claimant)

Authentication with symmetric key (cont.)

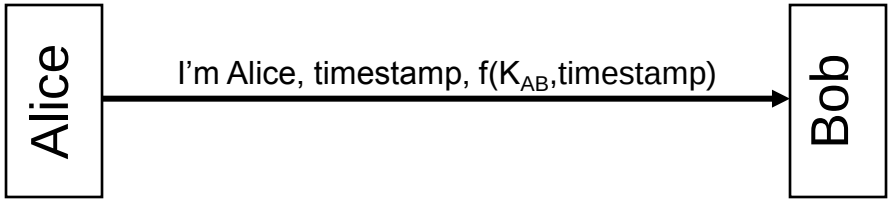


- R= D(K_{AB},X)
- differences:
  - **requires reversible cryptography**
  - **if R is a recognizable quantity with limited lifetime (e.g. a random number concatenated with a timestamp), Alice can authenticate Bob**
  - **if R is a recognizable quantity, Carol can mount an offline passwd-guessing attack without eavesdropping**
    - Carol obtains *E(K_{AB},R)* (second message) by just sending the first message to Bob claiming to be Alice
      - Carol doesn't need that Alice authenticates with Bob

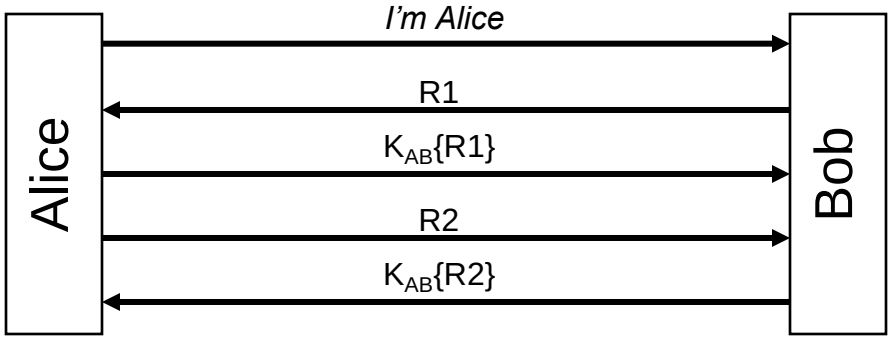
Authentication with symmetric key (cont.)



- differences:
  - **required only one message**
    - this mechanism can be added very easily to a protocol originally designed for sending passwd as cleartext
    - more efficient
  - **function f() does not require to be reversible**
  - **several pitfalls due to the time validity (time synchronization between Alice and Bob, authentication with multiple server with the same passwd, etc)**
- variant:



Mutual authentication with symmetric key

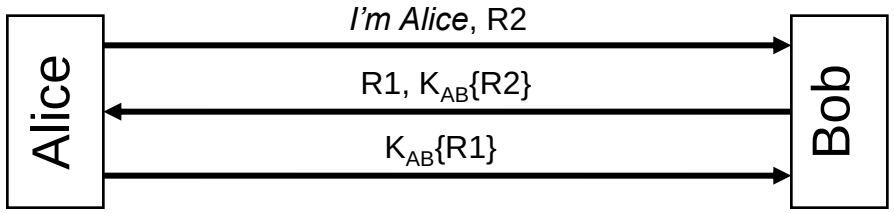


- or shorter:

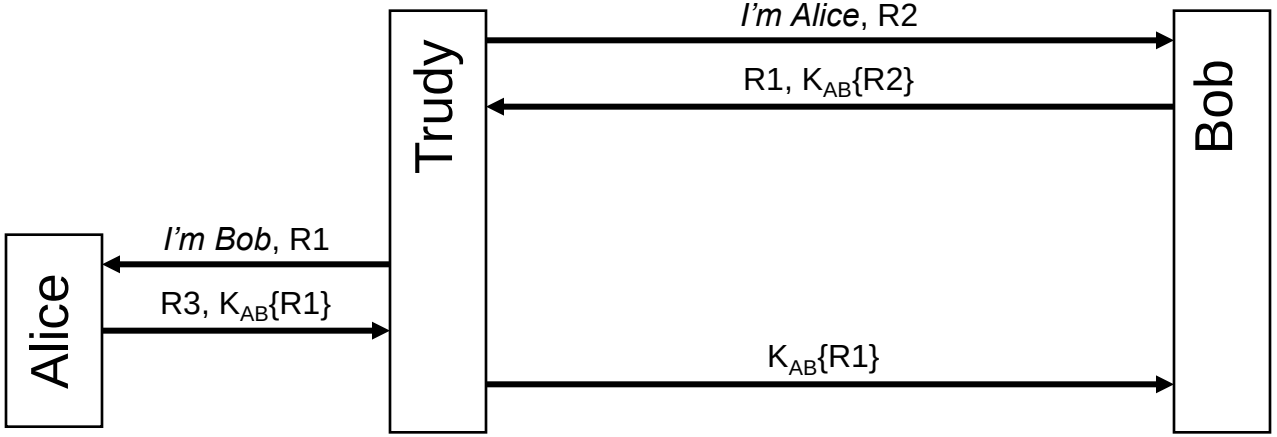


Mutual authentication with symmetric key

- or shorter:



- however, possible reflection attack:



- Good general principles of authentication protocols:
  - **the initiator should be the first to prove its identity**
  - **messages sent in opposite directions should differ**

Authentication with public key



- or



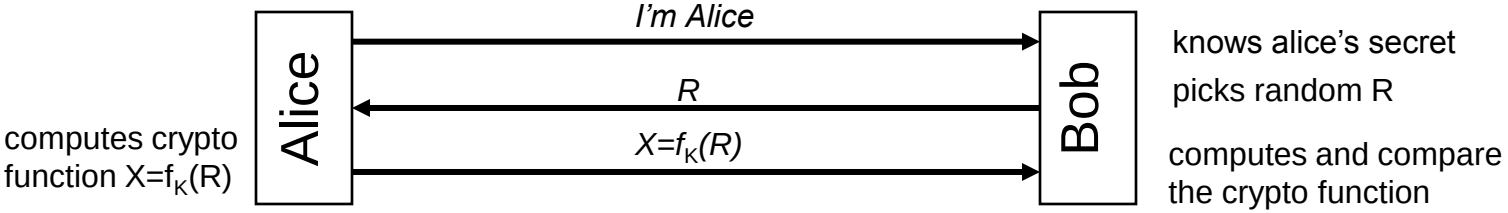
- property:
  - **the database at Bob is not required to be protected from reading**
    - must be protected only for unauthorized modification (integrity protection)
- drawbacks:
  - **if you can trick Alice into signing or decrypting something, you can impersonate Alice (first and second scheme, respectively)**
  - **by asking Alice to authenticate, you can obtain a signature or decryption**
- countermeasures:
  - **not use the same key for two different purposes unless the design for all uses are coordinated (this is a general rule), and/or**
  - **impose enough structure to be signed (nonce, realm, timestamp, etc.)**

Eavesdropping and server database reading

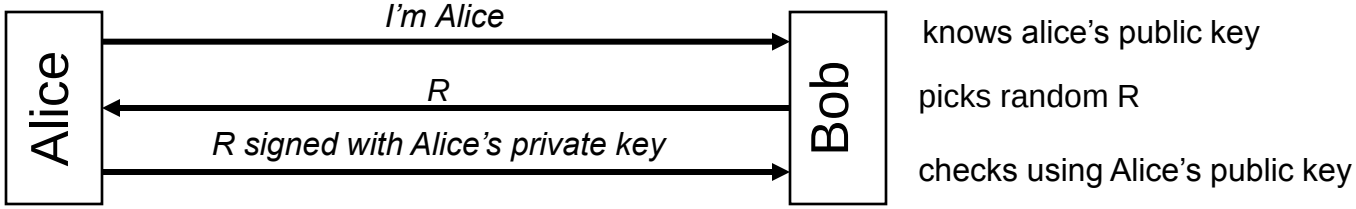
- Protection against server database reading:
  - **vulnerable to eavesdropping**



- Protection against eavesdropping:
  - **vulnerable to database reading, and to offline password guessing if the secret (key) is derived from a passwd, or offline brute force searching**



- Protection against both using asymmetric cryptography:



One-time passwords

- An alternative to use fixed secrets/passwords is using one-time secrets/passwords
  - **each password is used only once**
  - **such schemes are safe from passive adversaries who eavesdrop and later attempt impersonation**
- Can be easily implemented in Smart/token Cards



One-time passwords (cont.)

- Some one-time password variations:
  - **shared lists of one-time passwords**
    - use a sequence or set of secret passwords, (each valid for a single authentication), distributed as a pre-shared list
    - if the list is not used sequentially, the system may check the entered password against all remaining unused passwords
    - a variation involves use of a challenge-response table
    - a drawback is maintenance of the shared list
  - **sequentially updated one-time passwords**
    - during authentication using password *i*, the user creates and transmits to the system a new password (password *i+1*) encrypted under a key derived from password *i*
    - this method becomes difficult if communication failures occur
  - **one-time password sequences based on a one-way function**
    - more efficient than the previous one
    - may be viewed as a challenge-response protocol where challenge is implicitly defined by the current position within the pwd sequence

Example: HMAC-based One-Time Password (HOTP)

- The OTP code depends on a **counter** that increments each time you authenticate

$$\mathrm{HOTP}(K,C) = \mathrm{Trunc}(\mathrm{HMAC}(K,C))$$

  - **where:**
    - K: shared secret key
    - C: counter
      - integer, increases each use
    - Trunc: convert output to a 6–8 digit code

Example: Time-based OTP (TOTP)

- Replace counter with **time**

$$\mathrm{TOTP}(K,C) = \mathrm{Trunc}(\mathrm{HMAC}(K,T))$$

  - **where:**

$$T = \mathrm{floor}(\mathrm{current_time} / \mathrm{time_step})$$

  - **typical time_step: 30 seconds**

- Advantages:
  - **no counter to manage**
  - **stateless on server side**
  - **better usability**